

Федеральное государственное автономное образовательное
учреждение высшего образования «**Национальный
исследовательский университет ИТМО**»

Факультет Программной Инженерии и Компьютерной Техники

Лабораторная работа №6
по дисциплине «Вычислительная математика»

Вариант: 9

Преподаватель: Наумова
Надежда Александровна

Выполнил: Ровкова Анастасия Сергеевна
Группа: Р3216

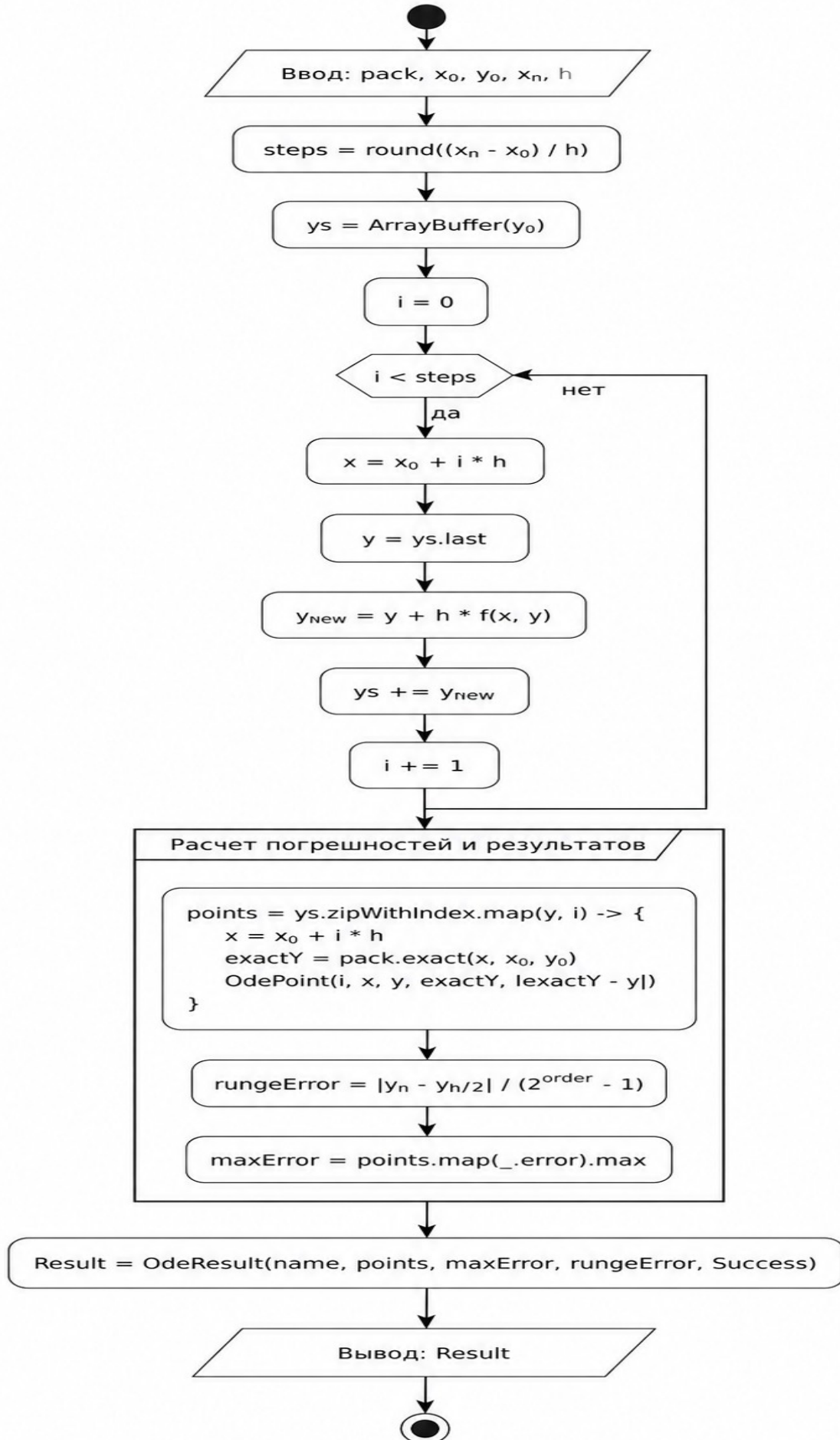
Санкт-Петербург, 2026 г

Цель работы

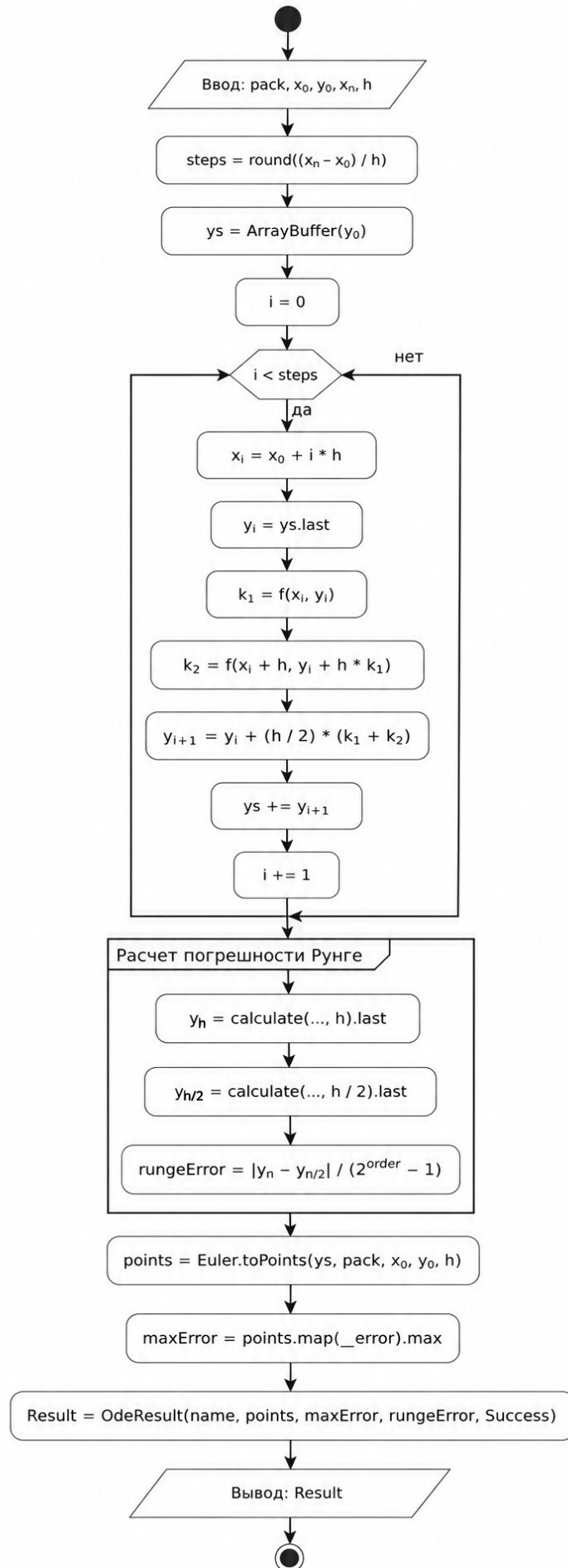
решить задачу Коши для обыкновенных дифференциальных уравнений на языке программирования SCALA.

Схемы:

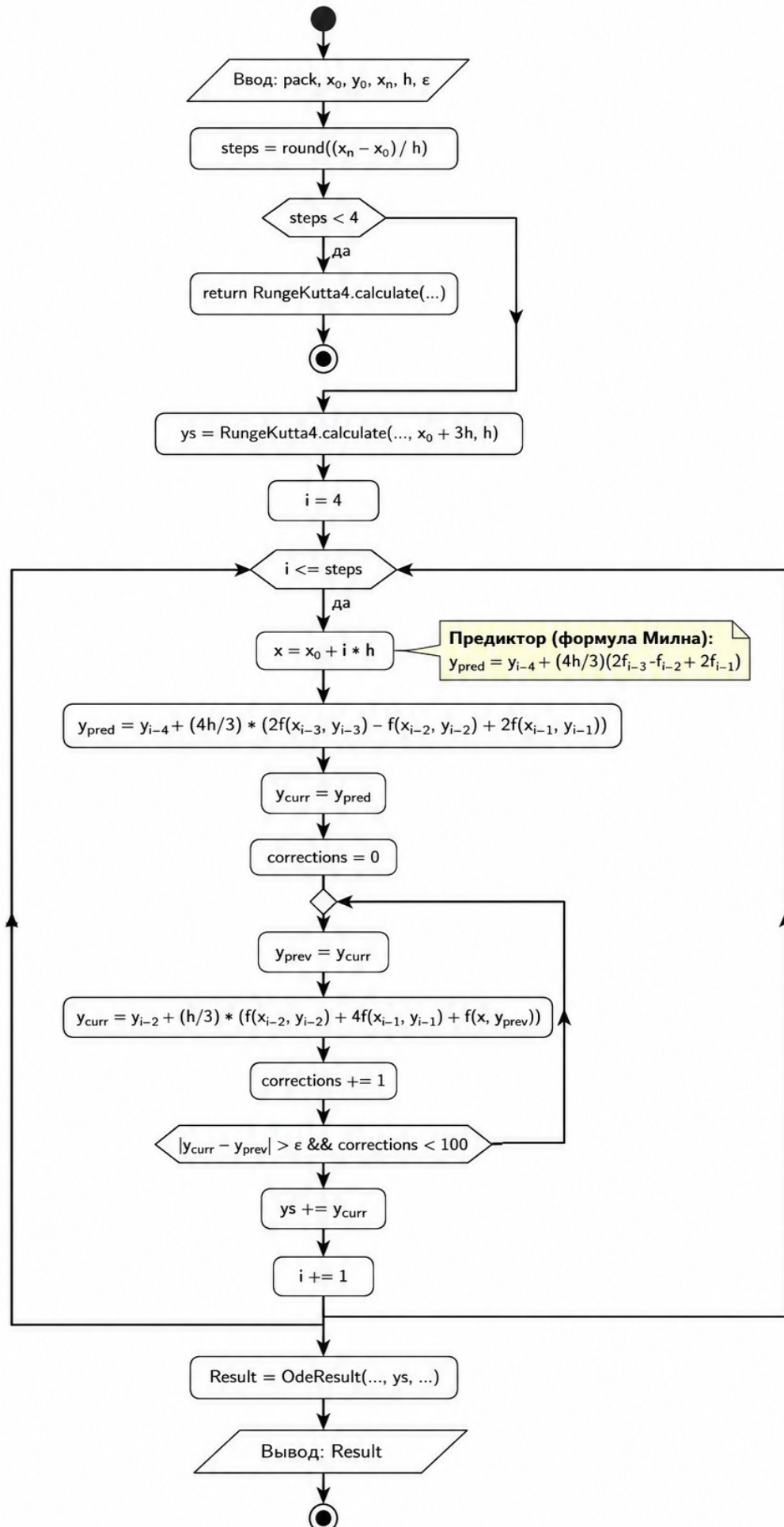
Алгоритм метода Эйлера



Алгоритм усовершенствованного метода Эйлера (RK2)



Алгоритм метода Милна



Код методов:

```
package solver.ode.methods
```

```
import solver.core.Message
```

```
import solver.ode.functions.{OdeMethod, OdePack}
```

```
import solver.ode.models.OdeResult
```

```
class Milne extends OdeMethod {
```

```
  override val name: String = "Метод Милна"
```

```
  override val order: Int = 4
```

```
  override def solve(pack: OdePack, x0: Double, y0: Double, xn: Double, h: Double, epsilon: Double): OdeResult = {
```

```
    val ys = Milne.calculate(pack.f, x0, y0, xn, h, epsilon)
```

```
    val points = Euler.toPoints(ys, pack, x0, y0, h)
```

```
    val maxError = if (points.isEmpty) 0 else points.map(_._error).max
```

```
    OdeResult(
```

```
      methodName = name,
```

```
      points = points,
```

```
      maxError = maxError,
```

```
      rungeError = 0,
```

```
      message = Message.Success
```

```
    )
```

```
  }
```

```
}
```

```
object Milne {
```

```
  private val MaxCorrections = 100
```

```
  def calculate(f: (Double, Double) => Double, x0: Double, y0: Double, xn: Double, h: Double, epsilon: Double): Seq[Double] = {
```

```
    val steps = math.round((xn - x0) / h).toInt
```

```
    if (steps < 4) return RungeKutta4.calculate(f, x0, y0, xn, h)
```

```
    val ys = scala.collection.mutable.ArrayBuffer.from(RungeKutta4.calculate(f, x0, y0, x0 + 3 * h, h))
```

```

for (i <- 4 to steps) {

  val x = x0 + i * h

  val f1 = f(x0 + (i - 3) * h, ys(i - 3))

  val f2 = f(x0 + (i - 2) * h, ys(i - 2))

  val f3 = f(x0 + (i - 1) * h, ys(i - 1))

  val predicted = ys(i - 4) + 4 * h / 3 * (2 * f1 - f2 + 2 * f3)

  var corrected = ys(i - 2) + h / 3 * (f2 + 4 * f3 + f(x, predicted))

  var previous = predicted

  var corrections = 0

  while (math.abs(corrected - previous) > epsilon && corrections < MaxCorrections) {

    previous = corrected

    corrected = ys(i - 2) + h / 3 * (f2 + 4 * f3 + f(x, previous))

    corrections += 1

  }

  ys += corrected

}

ys.toSeq

}

}

object RungeKutta4 {

  def calculate(f: (Double, Double) => Double, x0: Double, y0: Double, xn: Double, h: Double): Seq[Double] = {

    val steps = math.round((xn - x0) / h).toInt

    val ys = scala.collection.mutable.ArrayBuffer(y0)

    for (i <- 0 until steps) {

      val x = x0 + i * h

      val y = ys.last

      val k1 = h * f(x, y)

      val k2 = h * f(x + h / 2, y + k1 / 2)

      val k3 = h * f(x + h / 2, y + k2 / 2)

      val k4 = h * f(x + h, y + k3)

```

```
ys += y + (k1 + 2 * k2 + 2 * k3 + k4) / 6  
}
```

```
ys.toSeq  
  
}  
}
```

```
package solver.ode.methods
```

```
import solver.core.Message  
  
import solver.ode.functions.{OdeMethod, OdePack}  
  
import solver.ode.models.OdeResult
```

```
class ImprovedEuler extends OdeMethod {  
  
  override val name: String = "Усовершенствованный метод Эйлера"  
  
  override val order: Int = 2  
  
  
  override def solve(pack: OdePack, x0: Double, y0: Double, xn: Double, h: Double, epsilon: Double): OdeResult = {  
  
    val ys = ImprovedEuler.calculate(pack.f, x0, y0, xn, h)  
  
    val points = Euler.toPoints(ys, pack, x0, y0, h)  
  
    val rungeError = ImprovedEuler.rungeError(pack, x0, y0, xn, h, order)
```

```
  
    OdeResult(  
  
      methodName = name,  
  
      points = points,  
  
      maxError = if (points.isEmpty) 0 else points.map(_._error).max,  
  
      rungeError = rungeError,  
  
      message = Message.Success  
  
    )  
  
  }  
}
```

```
object ImprovedEuler {  
  
  def calculate(f: (Double, Double) => Double, x0: Double, y0: Double, xn: Double, h: Double): Seq[Double] = {  
  
    val steps = math.round((xn - x0) / h).toInt  
  
    val ys = scala.collection.mutable.ArrayBuffer(y0)
```



```

for (i <- 0 until steps) {

    val x = x0 + i * h

    val y = ys.last

    val predictor = y + h * f(x, y)

    val corrected = y + h / 2 * (f(x, y) + f(x + h, predictor))

    ys += corrected

}

```

```

ys.toSeq

```

```

}

```

```

def rungeError(pack: OdePack, x0: Double, y0: Double, xn: Double, h: Double, order: Int): Double = {

    val yH = calculate(pack.f, x0, y0, xn, h).lastOption.getOrElse(y0)

    val yHalf = calculate(pack.f, x0, y0, xn, h / 2).lastOption.getOrElse(y0)

    math.abs(yH - yHalf) / (math.pow(2, order) - 1)

}

}

```

```

package solver.ode.methods

```

```

import solver.core.Message

```

```

import solver.ode.functions.{OdeMethod, OdePack}

```

```

import solver.ode.models.{OdePoint, OdeResult}

```

```

class Euler extends OdeMethod {

```

```

    override val name: String = "Метод Эйлера"

```

```

    override val order: Int = 1

```

```

    override def solve(pack: OdePack, x0: Double, y0: Double, xn: Double, h: Double, epsilon: Double): OdeResult = {

```

```

        val ys = Euler.calculate(pack.f, x0, y0, xn, h)

```

```

        val points = Euler.toPoints(ys, pack, x0, y0, h)

```

```

        val rungeError = Euler.rungeError(pack, x0, y0, xn, h, order)

```

```

        OdeResult(

```

```

        methodName = name,

        points = points,

        maxError = if (points.isEmpty) 0 else points.map(_._error).max,

        rungeError = rungeError,

        message = Message.Success

    )

}

}

object Euler {

    def calculate(f: (Double, Double) => Double, x0: Double, y0: Double, xn: Double, h: Double): Seq[Double] = {

        val steps = math.round((xn - x0) / h).toInt

        val ys = scala.collection.mutable.ArrayBuffer(y0)

        for (i <- 0 until steps) {

            val x = x0 + i * h

            val y = ys.last

            ys += y + h * f(x, y)

        }

        ys.toSeq

    }

    def toPoints(ys: Seq[Double], pack: OdePack, x0: Double, y0: Double, h: Double): Seq[OdePoint] =

        ys.zipWithIndex.map { case (y, i) =>

            val x = x0 + i * h

            val exactY = pack.exact(x, x0, y0)

            OdePoint(i, x, y, exactY, math.abs(exactY - y))

        }

    def rungeError(pack: OdePack, x0: Double, y0: Double, xn: Double, h: Double, order: Int): Double = {

        val yH = calculate(pack.f, x0, y0, xn, h).lastOption.getOrElse(y0)

        val yHalf = calculate(pack.f, x0, y0, xn, h / 2).lastOption.getOrElse(y0)

        math.abs(yH - yHalf) / (math.pow(2, order) - 1)

    }

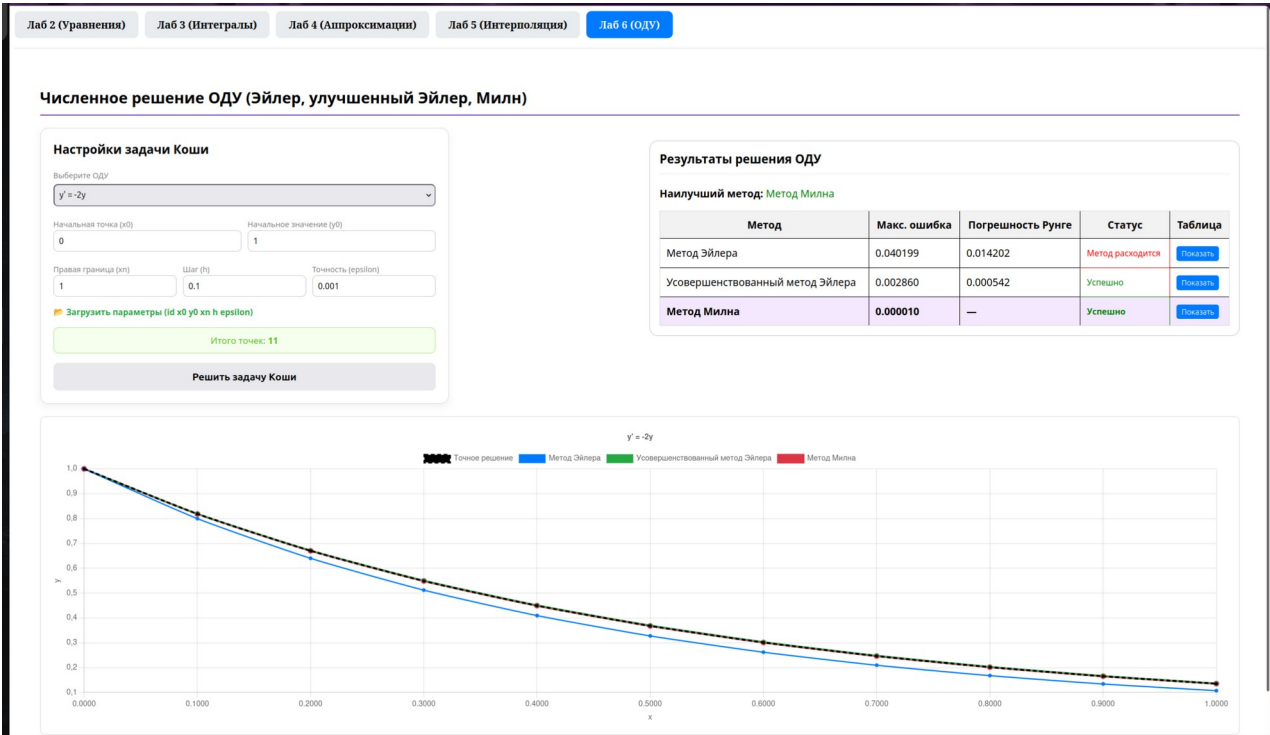
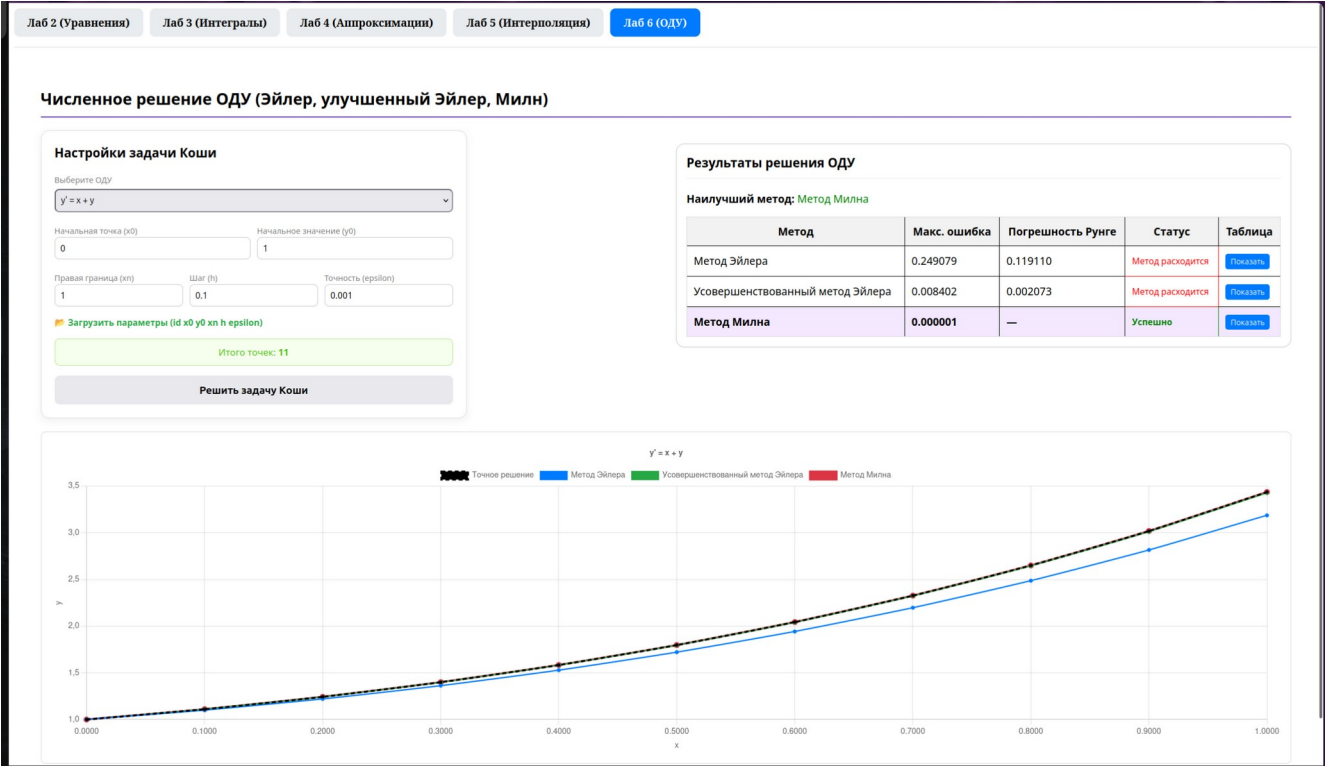
}

```

Листинг программы

https://github.com/Naswek/ITMO_labs/tree/main/
%D0%92%D1%8B%D1%87%D0%BC%D0%B0%D1%82/%D0%BB%D0%B0%D0%B1%20
2/backend/src/main/scala/solver/ode

Результаты выполнения программы при различных исходных данных.



Вывод

В результате выполнения лабораторной работы были изучены и реализованы основные методы нахождения частных решений дифф уравнений. Проведённые вычисления и тестирование программы показали корректность работы реализованных алгоритмов на различных наборах данных на языке программирования SCALA.